

A PROJECT REPORT ON

REAL-TIME TRAFFIC CONGESTION PREDICTION AND ROUTE OPTIMIZER

Domain: Smart City / Machine Learning

Submitted in partial fulfilment of the requirements for the award of the degree of

B.Sc. / B.E. / MCA – COMPUTER SCIENCE

Submitted by

STUDENT NAME

(Register No: XXXXXXXXX)

Under the guidance of

GUIDE NAME, M.Sc., M.Phil.,

Assistant Professor

DEPARTMENT OF COMPUTER SCIENCE

COLLEGE NAME

2025 – 2026

CONTENTS

CHAPTER	TITLE	PAGE
I	INTRODUCTION	1
	1.1 History of Domain	
	1.2 Overview of the Project	
	1.3 Motivation	
	1.4 Aim of the Project	
	1.5 Objectives	
	1.6 Literature Review	
II	SOFTWARE PROJECT PLAN	5
	2.1 Existing System	
	2.2 Proposed System	
	2.3 Project Schedule	
III	CUSTOMER REQUIREMENT DETERMINATION	7
	3.1 Functional Requirements	
	3.2 Non-functional Requirements	
	3.3 User Requirements	
	3.4 System Requirements	
IV	SOFTWARE REQUIREMENT SPECIFICATION	9
	4.1 Hardware Specification	
	4.2 Software Specification	
V	SYSTEM ANALYSIS	10
	5.1 Feasibility Study	

	5.2 About the Software	
VI	SYSTEM DESIGN	14
	6.1 UML Diagram	
	6.2 System Architecture	
	6.3 Data Flow Diagram	
	6.4 ER Diagram	
	6.5 Table Design	
	6.6 Algorithm Design	
	6.7 Modules	
VII	IMPLEMENTATION AND TESTING	23
	7.1 Introduction	
	7.2 Implementation	
	7.3 Test Cases	
	7.4 Source Code	
	7.5 Screen Layout	
VIII	CONCLUSION AND FUTURE ENHANCEMENT	30
	8.1 Conclusion	
	8.2 Future Enhancement	
IX	BIBLIOGRAPHY	31

CHAPTER I

INTRODUCTION

Rapid urbanisation and the growing number of private vehicles have made traffic congestion a daily problem in Indian cities and towns. Commuters lose many hours every week waiting at signals and junctions, which increases fuel consumption, air pollution and stress. Congestion becomes especially severe during school opening and closing hours, office peak hours, festivals, temple car festivals, cricket matches and rainy days.

Most navigation applications show the current traffic situation, but they do not predict how the traffic will change in the next 15 to 60 minutes. A route that looks clear now may become congested by the time the vehicle reaches it. Traffic police also lack predictive insight to decide where to deploy personnel in advance.

Machine learning models such as LSTM (Long Short-Term Memory) networks and gradient boosted trees can learn daily and weekly traffic patterns from historical data and combine them with live data, weather and calendar information to forecast travel time. This project, SmartRoute, predicts congestion on city roads 15 to 60 minutes ahead and recommends the fastest route using a graph algorithm in which each road is weighted by its predicted travel time.

LIMITATIONS

1. Static maps ignore upcoming congestion such as school hours or festivals.
2. Commuters waste fuel and time in traffic jams.
3. Traffic police lack predictive insight for deployment.
4. Existing route planners optimise only for current conditions.

1.1 History of Domain

Traffic engineering began with manual vehicle counts and fixed-time signals in the early twentieth century. Inductive loop detectors were introduced in the 1960s to count vehicles automatically. Statistical forecasting models such as ARIMA were applied to traffic flow in the 1970s and 1980s.

With the spread of GPS-enabled smartphones in the 2010s, crowd-sourced traffic data became available through services such as Google Maps and Waze. Deep learning models such as LSTM and graph neural networks (for example DCRNN, 2018) have since shown that traffic speed can be predicted accurately by learning both temporal and spatial dependencies of road networks.

1.2 Overview of the Project

SmartRoute collects speed and volume data for road segments from traffic APIs or simulated sensors. The data is cleaned and enriched with features such as hour of day, day of week, holiday or festival flag, school-hour flag and rainfall. An XGBoost model and an LSTM model are trained to predict the travel time of each segment for the next 15, 30 and 60 minutes.

The city road network is downloaded from OpenStreetMap using OSMnx and represented as a weighted directed graph. Edge weights are updated with predicted travel times for the expected time at which the vehicle will reach each road. The A* algorithm then finds the fastest route. The route and a colour-coded congestion map are displayed on a Leaflet map, and predictions are refreshed every five minutes.

1.3 Motivation

A commuter who knows that a road will be jammed in twenty minutes can choose another route now. Similarly, traffic police who know where congestion will build can act before it happens.

- **Motivation:** To move from reactive traffic information to predictive, time-aware route planning.
- **Outcome:** Shorter travel time, reduced fuel use and better deployment of traffic personnel.

1.4 Aim of the Project

The project is entitled as “**REAL-TIME TRAFFIC CONGESTION PREDICTION AND ROUTE OPTIMIZER**”. The aim of the project is to build a machine learning based system that predicts short-term traffic congestion on city roads and recommends the fastest route by applying graph search on a road network weighted with predicted travel times.

1.5 Objectives

- To collect and preprocess traffic speed data with time, weather and holiday features.
- To train LSTM and XGBoost models for 15-60 minute travel time prediction.
- To model the city road network as a weighted graph.
- To compute optimal routes with A*/Dijkstra using predicted weights.
- To display congestion forecasts on an interactive map for commuters and police.

1.6 Literature Review

Williams and Hoel (2003): Modelled vehicular traffic flow as a seasonal ARIMA process and showed that it provides good short-term forecasts for freeway traffic, establishing a classical baseline.

Ma et al. (2015): Applied LSTM neural networks to traffic speed prediction using microwave detector data from Beijing and showed that LSTM outperforms ARIMA, SVR and other neural networks.

Li et al. (2018): Proposed the Diffusion Convolutional Recurrent Neural Network (DCRNN) which models traffic as a diffusion process on a directed graph and achieved state-of-the-art results on the METR-LA and PEMS-BAY datasets.

Chen and Guestrin (2016): Introduced XGBoost, a scalable tree boosting system that is widely used for tabular prediction tasks due to its speed and accuracy.

Hart, Nilsson and Raphael (1968): Introduced the A* search algorithm which finds the shortest path in a graph using a heuristic estimate of the remaining distance.

Boeing (2017): Presented OSMnx, a Python package to download, model and analyse street networks from OpenStreetMap.

CHAPTER II

SOFTWARE PROJECT PLAN

2.1 Existing System

Existing navigation applications compute routes based on current or recently observed traffic speeds. Traffic police depend on experience and wireless reports from junctions. Signal timings are mostly fixed. No public tool forecasts congestion for local events such as school hours or temple festivals.

DISADVANTAGES

- Routes are optimal only for current conditions.
- No forecast of congestion build-up.
- Local events and weather are not considered.
- Police deployment is reactive.
- Commercial APIs are costly for large-scale use by local bodies.

2.2 Proposed System

The proposed SmartRoute system predicts travel time for each road segment for the next hour using LSTM and XGBoost models trained on historical and live data enriched with weather, holiday and school-hour features. A time-dependent A* search uses the predicted travel times to recommend the fastest route, and a congestion forecast map helps traffic police plan deployment.

ADVANTAGES

- Predicts congestion up to 60 minutes ahead.
- Time-aware routing avoids roads that will become congested.
- Considers festivals, school timings and rain.
- Helps police plan in advance.
- Uses free OpenStreetMap data.

2.3 Project Schedule

The development of SmartRoute follows the iterative waterfall model. Each phase produces a deliverable that is reviewed before the next phase begins, and feedback from testing is fed back into design and implementation. The planned schedule is shown below.

S.No	Phase	Duration
1	Requirement analysis and problem study	Week 1 - 2
2	Literature survey and existing system study	Week 3 - 4
3	Dataset collection / hardware procurement	Week 5 - 6
4	System design (DFD, UML, ER, table design)	Week 7 - 8
5	Implementation of modules	Week 9 - 12
6	Testing and model evaluation	Week 13 - 14
7	Documentation and final review	Week 15 - 16

CHAPTER III

CUSTOMER REQUIREMENT DETERMINATION

3.1 Functional Requirements

Functional requirements define the specific features and operations a system must perform to meet business and user needs. They describe what the system should do and how it interacts with users or other systems. They focus on the behaviour and functionality of the system and represent features that can be directly observed and tested in the final product. The functional requirements of the SmartRoute system are listed below.

- **Data collection:** Collect segment speed and volume data every 5 minutes.
- **Feature engineering:** Add time, weather, holiday and event features.
- **Prediction:** Predict travel time for 15, 30 and 60 minutes.
- **Graph building:** Build the road network graph from OpenStreetMap.
- **Route optimisation:** Compute the fastest route using predicted edge weights.
- **Map display:** Show route and colour-coded congestion.
- **Police view:** Show predicted hotspots and suggested deployment.
- **Re-evaluation:** Refresh predictions every 5 minutes.

3.2 Non-functional Requirements

Non-functional requirements determine the performance standards and quality attributes of software, e.g. system usability, effectiveness, security, scalability, etc. While functional requirements determine what the system does, non-functional requirements describe HOW the system does it. If an application does not meet non-functional requirements, it continues to perform its basic functions; however, it will not be able to provide a good user experience. Therefore, non-functional requirements are as important as functional requirements for successful product adoption.

- **Performance:** Route computed within 2 seconds.
- **Accuracy:** Mean absolute percentage error below 15%.
- **Availability:** Service available during peak hours without failure.

- **Scalability:** Supports a city graph of 50,000 edges.
- **Usability:** Map-based interface usable on mobile.
- **Caching:** Redis caching of predictions to reduce load.

3.3 User Requirements

- Enter source and destination easily
- Get the fastest route with expected arrival time
- See which roads will be congested
- Choose departure time in the future
- View on mobile while travelling

3.4 System Requirements

- Python ML server with Flask API
- Redis cache for predictions
- Road network graph from OSMnx
- Historical traffic dataset
- Leaflet map front end

CHAPTER IV

SOFTWARE REQUIREMENT SPECIFICATION

4.1 Hardware Specification

System : Intel i5 Processor 2.4 GHz or above
Hard Disk : 256 GB SSD
RAM : 16 GB
GPU : Optional NVIDIA GPU for LSTM training
Monitor : 15" colour monitor

4.2 Software Specification

Operating System : Windows 10 / Ubuntu 22.04
Coding Language : Python 3.10, JavaScript
ML Libraries : scikit-learn, XGBoost, PyTorch
Graph Library : OSMnx, NetworkX
Back End : Flask
Cache : Redis
Front End : HTML, Leaflet.js
Dataset : METR-LA / city traffic API / simulated data

CHAPTER V

SYSTEM ANALYSIS

5.1 Feasibility Study

A feasibility study is carried out to find whether the proposed SmartRoute system is worth developing. Three key considerations are involved in the feasibility analysis: technical, economic and operational feasibility.

TECHNICAL FEASIBILITY

Open-source libraries for machine learning (XGBoost, PyTorch) and graph processing (OSMnx, NetworkX) are mature and well documented. Public traffic datasets such as METR-LA are available for training and validation. The system is technically feasible.

ECONOMIC FEASIBILITY

OpenStreetMap data is free, and the server can run on a modest cloud instance. No paid map API is mandatory. The system is economically feasible.

OPERATIONAL FEASIBILITY

The commuter interface works like familiar map applications, and the police dashboard shows a simple hotspot map. Little training is required, so it is operationally feasible.

5.2 About the Software

PYTHON

Python is a high-level, interpreted, general-purpose programming language created by Guido van Rossum and first released in 1991. Its design philosophy emphasises code readability through significant indentation, and its simple syntax allows developers to express concepts in fewer lines of code than languages such as C++ or Java.

Python supports multiple programming paradigms including procedural, object-oriented and functional programming. It has a very large standard library and an ecosystem of third-party packages available through the Python Package Index (PyPI), which makes it the most popular language for data science, machine learning, automation and back-end web development.

- Easy to learn, read and maintain
- Huge library support for AI, ML and data processing (NumPy, Pandas, scikit-learn)
- Platform independent - runs on Windows, Linux and macOS
- Strong community support and documentation

XGBOOST

XGBoost (Extreme Gradient Boosting) is an optimised, distributed gradient boosting library designed to be highly efficient, flexible and portable. It implements machine learning algorithms under the gradient boosting framework, where many shallow decision trees are added one after another and each new tree corrects the errors of the previous ones.

XGBoost handles missing values automatically, supports regularisation to avoid over-fitting, and allows class weighting through the `scale_pos_weight` parameter, which makes it very suitable for imbalanced problems such as fraud and fault detection on tabular data.

- High accuracy on tabular data
- Fast training with parallel tree construction
- Feature importance for explainability

PYTORCH

PyTorch is an open-source deep learning framework originally developed by Meta AI. It uses dynamic computation graphs, which means the network is built at run time; this makes debugging simple and the code feels like normal Python.

PyTorch is widely used for research and production in computer vision and natural language processing. The `torch.nn` module provides layers such as LSTM, GRU and convolution, and models can be exported using TorchScript or ONNX for deployment.

- Dynamic graphs and pythonic debugging
- Rich ecosystem (TorchVision, TorchText, HuggingFace)
- GPU acceleration through CUDA

SCIKIT-LEARN

Scikit-learn is a free machine learning library for Python built on NumPy, SciPy and Matplotlib. It provides simple and efficient tools for predictive data analysis including classification, regression, clustering and dimensionality reduction.

It contains implementations of popular algorithms such as Random Forest, Support Vector Machine, Logistic Regression, k-Nearest Neighbours and Isolation Forest, along with utilities for train-test splitting, cross-validation, feature scaling, pipelines and evaluation metrics such as accuracy, precision, recall and F1-score.

- Consistent fit / predict API for all models
- Built-in preprocessing and model selection tools
- Well documented and production tested

OSMNX AND NETWORKX

OSMnx is a Python package that downloads street networks from OpenStreetMap and models them as NetworkX graphs, where intersections are nodes and road segments are edges with attributes such as length and speed limit.

NetworkX provides graph algorithms such as Dijkstra and A* shortest path, which are used to compute the best route once edge weights are updated with predicted travel times.

FLASK

Flask is a lightweight WSGI web application framework written in Python. It is classified as a micro-framework because it does not require particular tools or libraries; it has no database abstraction layer or form validation built in, but extensions can add these features as needed.

Flask provides URL routing, request handling, Jinja2 templating and a development server. Because it is small and simple, it is ideal for exposing a trained machine learning model as a REST API or building a small web application quickly.

- Minimal and easy to learn
- Jinja2 templates for HTML pages
- Large set of extensions (Flask-Login, Flask-SQLAlchemy, Flask-CORS)

REDIS

Redis is an open-source, in-memory data structure store used as a database, cache and message broker. It supports strings, hashes, lists, sets, sorted sets and streams.

Because all data is kept in memory, Redis offers sub-millisecond response times, which makes it suitable for caching predictions, counting events in sliding windows and passing real-time messages between services.

LEAFLET.JS

Leaflet is the leading open-source JavaScript library for mobile-friendly interactive maps. It is lightweight (about 42 KB) and has all the mapping features most developers need, such as tile layers, markers, pop-ups, polylines and GeoJSON layers.

Leaflet works with free OpenStreetMap tiles, so maps can be shown without paying for a commercial map API.

CHAPTER VI

SYSTEM DESIGN

System design is the process of defining the architecture, components, modules, interfaces and data of a system to satisfy the specified requirements. The design of SmartRoute is presented using UML use case diagram, system architecture, data flow diagrams, ER diagram and table design.

6.1 UML Diagram

USE CASE DIAGRAM

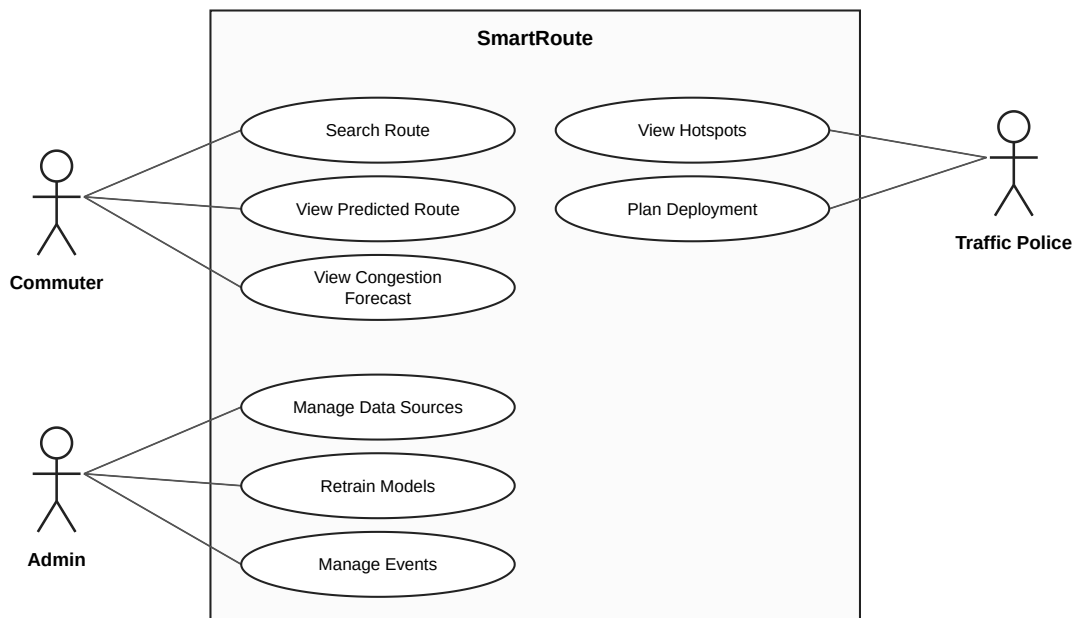


Fig 1: Use case diagram of SmartRoute

6.2 System Architecture

SmartRoute is organised into a data ingestion layer, a prediction layer, a routing layer and a presentation layer. Predictions are cached in Redis so that many route requests can be served quickly.

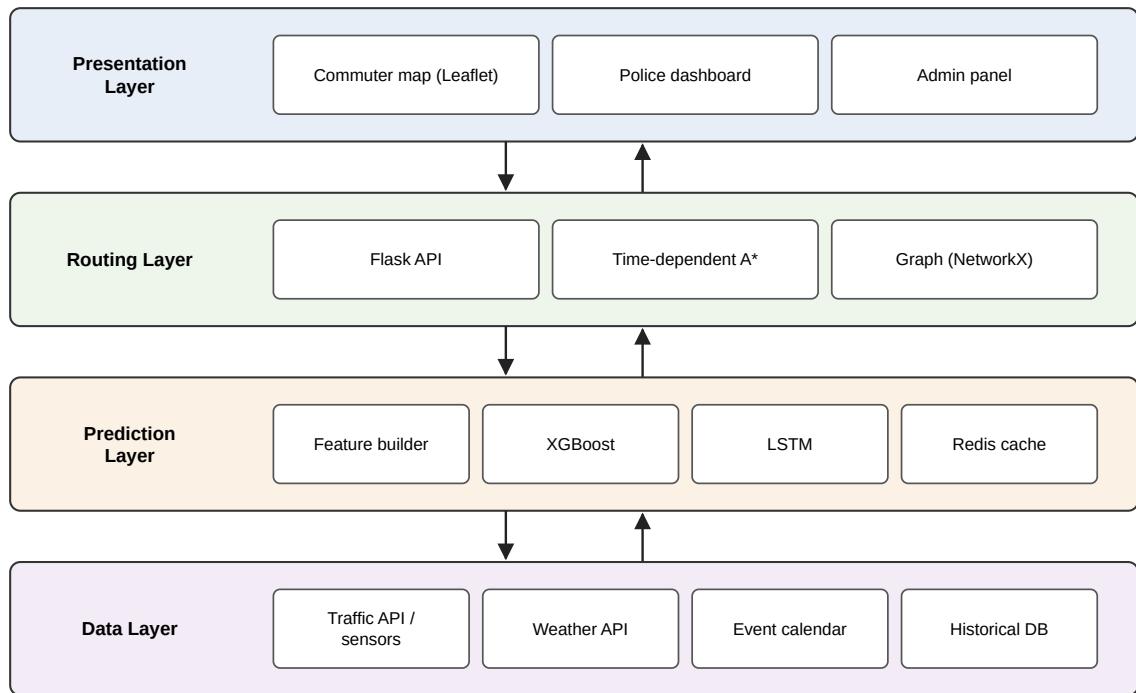


Fig 2: System architecture

6.3 Data Flow Diagram

A Data Flow Diagram (DFD) is a graphical representation of the flow of data through an information system. The Level 0 DFD (context diagram) shows the whole system as a single process with its external entities, and the Level 1 DFD breaks it into its major processes and data stores.

LEVEL 0 DFD (CONTEXT DIAGRAM)

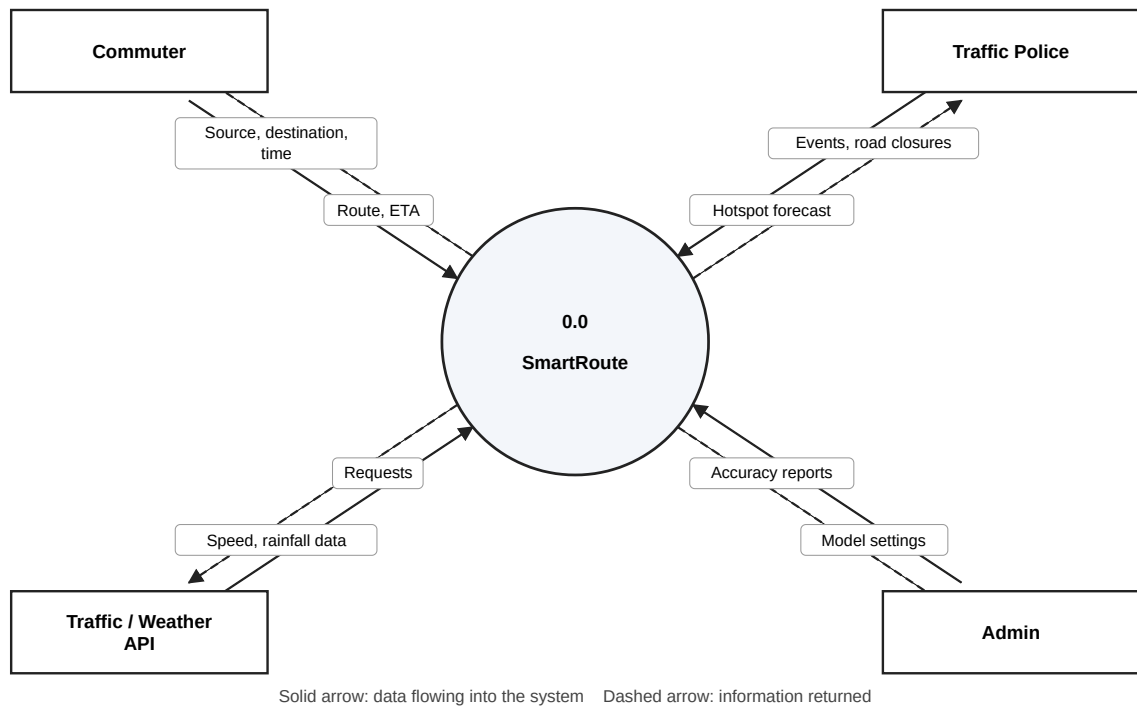


Fig 3: Level 0 data flow diagram

LEVEL 1 DFD

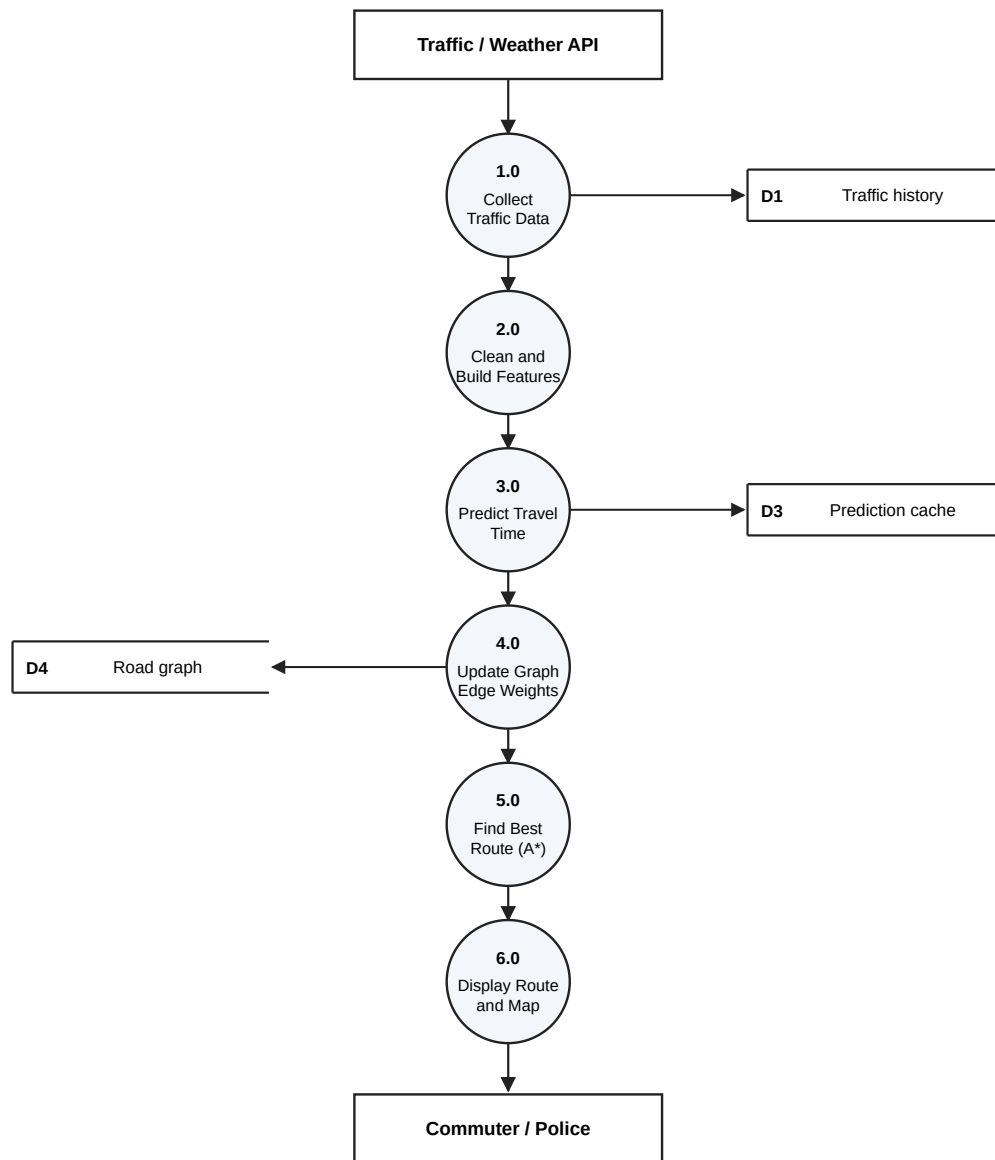


Fig 4: Level 1 data flow diagram

6.4 ER Diagram

The Entity Relationship (ER) diagram shows the entities stored in the database, their key attributes and the relationships between them. 1:N indicates a one-to-many relationship.

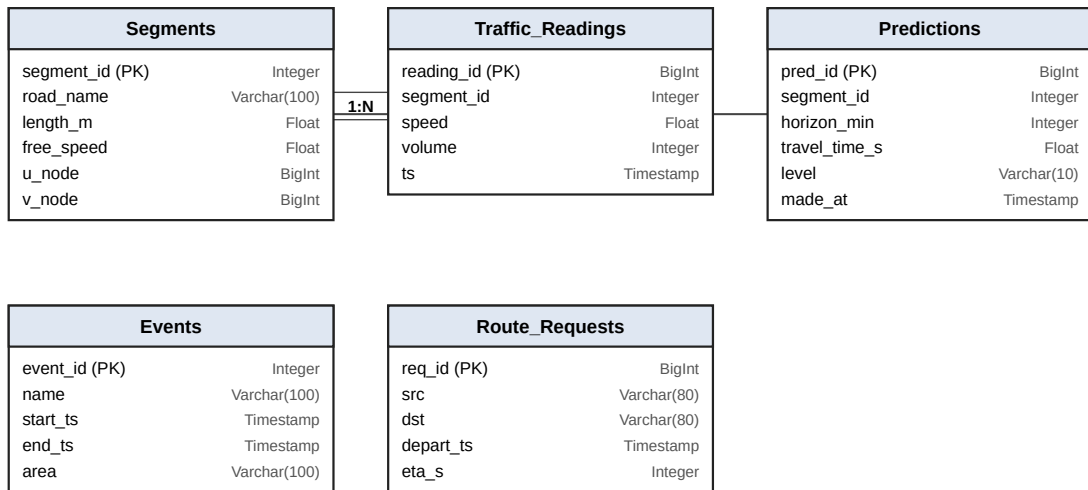


Fig 5: Entity relationship diagram

6.5 Table Design

SEGMENTS

FIELD NAME	DATA TYPE	DESCRIPTION
segment_id	Integer	Primary key (OSM edge id)
road_name	Varchar(100)	Road name
length_m	Float	Length in metres
free_speed	Float	Free-flow speed km/h
u_node	BigInt	Start node
v_node	BigInt	End node

TRAFFIC_READINGS

FIELD NAME	DATA TYPE	DESCRIPTION
reading_id	BigInt	Primary key
segment_id	Integer	Foreign key to Segments
speed	Float	Observed speed km/h
volume	Integer	Vehicles per 5 min
ts	Timestamp	Time

PREDICTIONS

FIELD NAME	DATA TYPE	DESCRIPTION
pred_id	BigInt	Primary key
segment_id	Integer	Foreign key to Segments
horizon_min	Integer	15 / 30 / 60
travel_time_s	Float	Predicted travel time
level	Varchar(10)	low / medium / high
made_at	Timestamp	Prediction time

EVENTS

FIELD NAME	DATA TYPE	DESCRIPTION
event_id	Integer	Primary key
name	Varchar(100)	Festival / event
start_ts	Timestamp	Start
end_ts	Timestamp	End
area	Varchar(100)	Affected area

ROUTE_REQUESTS

FIELD NAME	DATA TYPE	DESCRIPTION
req_id	BigInt	Primary key
src	Varchar(80)	Source
dst	Varchar(80)	Destination
depart_ts	Timestamp	Departure time
eta_s	Integer	Predicted travel time

6.6 Algorithm Design

The system combines supervised travel-time prediction with time-dependent shortest path search.

1. **Collect speed/volume data from APIs or simulated sensors** – Data is collected every 5 minutes for each road segment.
2. **Clean data and add time, weather, holiday features** – Missing values are interpolated, outliers removed, and features such as hour, weekday, rain_mm, is_holiday, school_hour and lag speeds are added.
3. **Train LSTM/XGBoost model for travel-time prediction** – XGBoost uses lag features; LSTM uses the last 12 readings (1 hour) as a sequence. The better model per horizon is selected.
4. **Build road network as weighted graph** – OSMnx downloads the drivable network; nodes are junctions and edges are road segments.
5. **Update edge weights with predicted values** – Edge weight = predicted travel time for the time at which the vehicle is expected to reach that edge.
6. **Run A* / Dijkstra for best route** – A* uses straight-line distance divided by maximum speed as an admissible heuristic.
7. **Display route and congestion map** – The route polyline and coloured segments are drawn using Leaflet.
8. **Re-evaluate every few minutes** – Predictions are refreshed every 5 minutes and active routes are recomputed.

PSEUDO CODE

```

FUNCTION BEST_ROUTE(G, src, dst, t0)
  open <- PRIORITY_QUEUE((h(src), src, t0)); g[src] <- 0
  WHILE open NOT EMPTY
    (f, u, t) <- POP(open)
    IF u = dst THEN RETURN PATH(u), g[u]
    FOR each edge (u, v):
      h_min <- NEAREST_HORIZON(t - now)           // 15, 30 or 60 min
      w <- PREDICTED_TIME(edge, h_min)
      IF g[u] + w < g[v] THEN
        g[v] <- g[u] + w; parent[v] <- u
        PUSH(open, (g[v] + h(v), v, t + w))
  RETURN no route

```

6.7 Modules

The modules are organised by the three users of the system: Commuter, Traffic Police and Admin.

MODULES

COMMUTER

1. SEARCH ROUTE
2. VIEW PREDICTED ROUTE
3. VIEW CONGESTION FORECAST

TRAFFIC POLICE

1. VIEW HOTSPOTS
2. PLAN DEPLOYMENT

ADMIN

1. MANAGE DATA SOURCES
2. RETRAIN MODELS
3. MANAGE EVENTS

COMMUTER

SEARCH ROUTE

The commuter enters the source, destination and departure time (now or later). Locations can be selected on the map or typed.

VIEW PREDICTED ROUTE

The fastest route is drawn on the map with the expected travel time and arrival time, and alternative routes are listed.

VIEW CONGESTION FORECAST

A colour-coded map shows predicted congestion (green, orange, red) for the next 15, 30 and 60 minutes.

TRAFFIC POLICE

VIEW HOTSPOTS

Shows junctions where congestion is predicted to become severe in the next hour.

PLAN DEPLOYMENT

Suggests junctions that need personnel and allows marking of road closures and events.

ADMIN

MANAGE DATA SOURCES

Admin configures traffic API keys, sensor feeds and weather source.

RETRAIN MODELS

Admin retrains the prediction models with new data and compares accuracy.

MANAGE EVENTS

Admin enters festival, school and event calendars used as features.

CHAPTER VII

IMPLEMENTATION AND TESTING

7.1 Introduction

SmartRoute was implemented in Python. The prediction models were trained in Jupyter notebooks, and the Flask server loads the trained models and the city graph at start-up. A background scheduler runs every 5 minutes to fetch new data, predict travel times for all segments and store the results in Redis.

7.2 Implementation

For development and evaluation the public METR-LA dataset (207 sensors, 5-minute speed readings) was used, together with simulated data for a local town network generated from OSMnx with time-of-day speed profiles and event effects. XGBoost was trained with 300 trees and max depth 6; the LSTM had two layers with 64 units.

Testing is a process of running software with the intention of finding errors. The software which has been developed has to be tested to prove its validity. Testing is considered to be the least creative phase of the whole cycle of system design; in the real sense it is the phase which helps to bring out the creativity of the other phases. Testing a program is the first step in the debugging process.

A good testing suite is vital because all possible execution paths inside the code must be tested in order to have any hope that it is bug free. Testing of the system is done to ensure the integrity of the system. The aim of testing is to prove that the developed system addresses the predefined processing requirements and will perform reliably and efficiently during operation.

PURPOSES OF TESTING

- To affirm the quality of the project.
- To find and eliminate any errors from previous stages.
- To validate the software against its requirements.

- To ensure the reliability of the system.

UNIT TESTING

In the lines of this strategy all the individual functions and modules were put to the test independently. By following this strategy all the errors in coding were identified and corrected. This method was applied in combination with white box and black box testing techniques to find the errors in each module.

INTEGRATION TESTING

This testing strategy has two different approaches, namely the top-down approach, in which integration is carried out from the top-level module to the bottom, and the bottom-up approach, in which integration is carried out from the low-level modules to the top. The modules were tested using the bottom-up approach by introducing stubs for the top-level functions. This test was used to identify errors in the interfaces and in passing parameters between functions.

VALIDATION TESTING

The main aim of this testing is to verify that the software system does what it was designed for. Alpha testing and beta testing were conducted to ensure the validity of the system. Alpha testing was done at the developer's site and beta testing was carried out by sample end users.

SYSTEM TESTING

The idea of applying this strategy was to put the system through a series of different tests whose primary purpose is to fully exercise the computer-based system. Although each test has a different purpose, all work to verify that all system elements have been properly integrated and perform their allocated functions. Security testing and performance testing were also conducted.

ACCEPTANCE TESTING

Acceptance testing involves planning and execution of functional tests, performance tests and stress tests to verify that the implemented system satisfies its

requirements. Typically, the acceptance test incorporates test cases developed during unit testing and integration testing.

7.3 Test Cases

Test ID	Test Description	Input	Expected Output	Result
TC01	Route between two points	Bus stand to railway station	Route drawn with ETA	Pass
TC02	Future departure	Depart at 08:45 (school hour)	Route avoids school road	Pass
TC03	Invalid location	Unknown place name	Error 'Location not found'	Pass
TC04	Prediction refresh	Wait 5 minutes	New predictions in cache	Pass
TC05	Festival event	Event added on temple road	High congestion predicted on that road	Pass
TC06	Hotspot view	Police login	Top 5 predicted hotspots listed	Pass
TC07	Rain feature	Rainfall 20 mm	Travel times increased	Pass
TC08	Performance	100 route requests	Average response < 2 s	Pass

PERFORMANCE EVALUATION

Prediction accuracy on the test period for the 30-minute horizon:

Model	MAE (km/h)	RMSE (km/h)	MAPE
Historical average	4.16	7.80	13.0%
ARIMA	3.60	6.90	10.7%
XGBoost	3.21	6.35	9.1%
LSTM (proposed)	3.05	6.12	8.6%

7.4 Source Code

train_xgb.py

```
import pandas as pd, xgboost as xgb, joblib
from sklearn.metrics import mean_absolute_error

df = pd.read_parquet("data/traffic_5min.parquet")           # segment_id, ts, speed,
```

```

rain_mm
df["hour"] = df.ts.dt.hour; df["dow"] = df.ts.dt.dayofweek
df["is_holiday"] =
df.ts.dt.date.isin(pd.read_csv("data/holidays.csv").date).astype(int)
df["school_hour"] = df.hour.isin([8, 9, 15, 16]).astype(int) & (df.dow < 5)
df = df.sort_values(["segment_id", "ts"])
for lag in range(1, 7):
    df[f"lag{lag}"] = df.groupby("segment_id").speed.shift(lag)
for h in (3, 6, 12):                                     # 15, 30, 60 minutes
    ahead
    df[f"y{h}"] = df.groupby("segment_id").speed.shift(-h)
df = df.dropna()

FEATS = ["hour", "dow", "is_holiday", "school_hour", "rain_mm"] + [f"lag{i}" for
i in range(1, 7)]
split = df.ts.quantile(0.8)
train, test = df[df.ts < split], df[df.ts >= split]
for h in (3, 6, 12):
    m = xgb.XGBRegressor(n_estimators=300, max_depth=6, learning_rate=0.05,
subsample=0.8)
    m.fit(train[FEATS], train[f"y{h}"])
    print(h * 5, "min MAE:", mean_absolute_error(test[f"y{h}"]),
m.predict(test[FEATS]))
    joblib.dump(m, f"models/xgb_{h*5}.pkl")

```

router.py

```

import heapq, math, time, json
import osmnx as ox, redis
from flask import Flask, request, jsonify

app = Flask(__name__)
G = ox.load_graphml("data/city.graphml")
cache = redis.Redis()
VMAX = 60 / 3.6                                     # m/s for admissible
heuristic

def predicted_time(u, v, k, minutes_ahead):
    horizon = 15 if minutes_ahead <= 15 else 30 if minutes_ahead <= 30 else 60
    speed = cache.hget(f"pred:{horizon}", f"{u}-{v}")
    data = G[u][v][k]
    kmph = float(speed) if speed else data.get("speed_kph", 30)
    return data["length"] / (max(kmph, 3) / 3.6)

def heuristic(a, b):
    ya, xa, yb, xb = G.nodes[a]["y"], G.nodes[a]["x"], G.nodes[b]["y"],
G.nodes[b]["x"]
    return ox.distance.great_circle(ya, xa, yb, xb) / VMAX

def astar(src, dst, depart):
    g, parent = {src: 0}, {}
    heap = [(heuristic(src, dst), src)]
    while heap:
        _, u = heapq.heappop(heap)
        if u == dst:
            path = [u]
            while u in parent: u = parent[u]; path.append(u)
            return path[::-1], g[dst]
        for v, edges in G[u].items():
            for k in edges:
                ahead = (depart + g[u] - time.time()) / 60

```

```

        w = predicted_time(u, v, k, ahead)
        if g[u] + w < g.get(v, math.inf):
            g[v] = g[u] + w; parent[v] = u
            heapq.heappush(heap, (g[v] + heuristic(v, dst), v))

    return None, None

@app.route("/route")
def route():
    s = ox.distance.nearest_nodes(G, float(request.args["slng"]),
float(request.args["slat"]))
    d = ox.distance.nearest_nodes(G, float(request.args["dlng"]),
float(request.args["dlat"]))
    depart = float(request.args.get("depart", time.time()))
    path, secs = astar(s, d, depart)
    if not path:
        return jsonify(error="No route found"), 404
    coords = [[G.nodes[n]["y"], G.nodes[n]["x"]] for n in path]
    return jsonify(route=coords, eta_min=round(secs / 60, 1))

```

7.5 Screen Layout

SCREENSHOTS:

ROUTE SEARCH

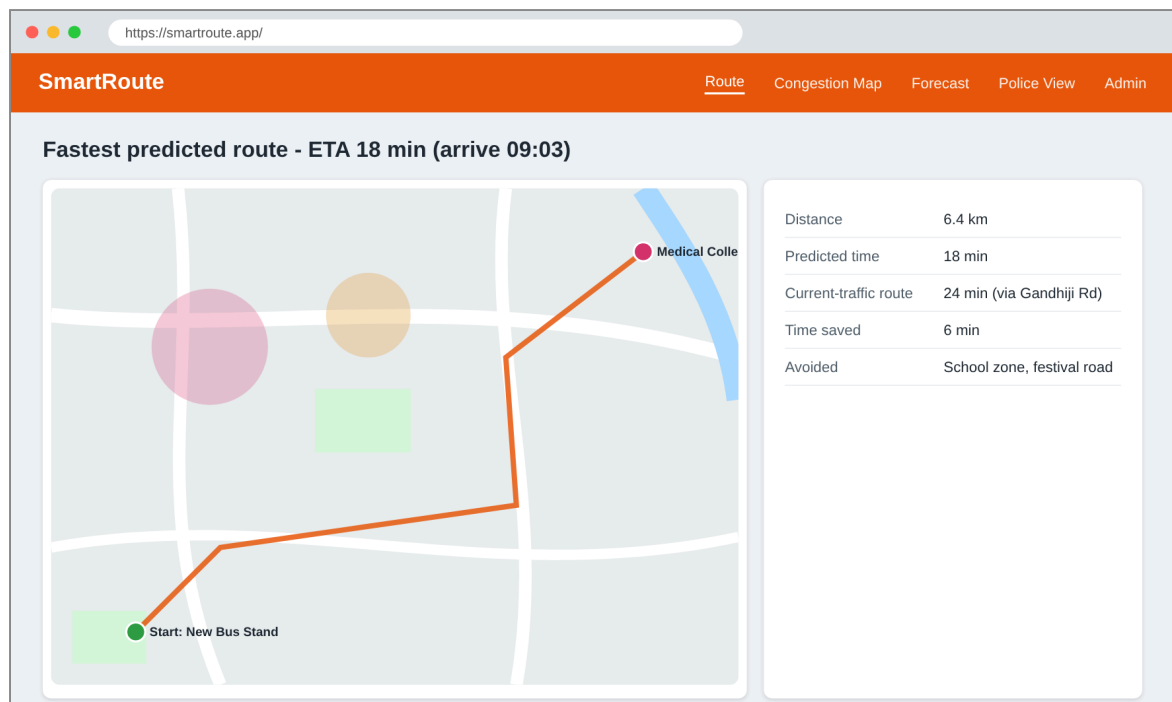
The screenshot displays the SmartRoute web application interface. The browser address bar shows `https://smartroute.app/`. The application has an orange header bar with the "SmartRoute" logo on the left and navigation links "Route", "Congestion Map", "Forecast", "Police View", and "Admin" on the right. The "Route" link is currently selected.

The main content area is titled "Plan your trip" and contains a form for route planning. The form has four input fields: "From" (containing "New Bus Stand, Thanjavur"), "To" (containing "Medical College Hospital"), "Departure" (containing "Today 08:45 AM"), and "Optimise for" (containing "Fastest (predicted)"). Below these fields is an orange "Find Route" button.

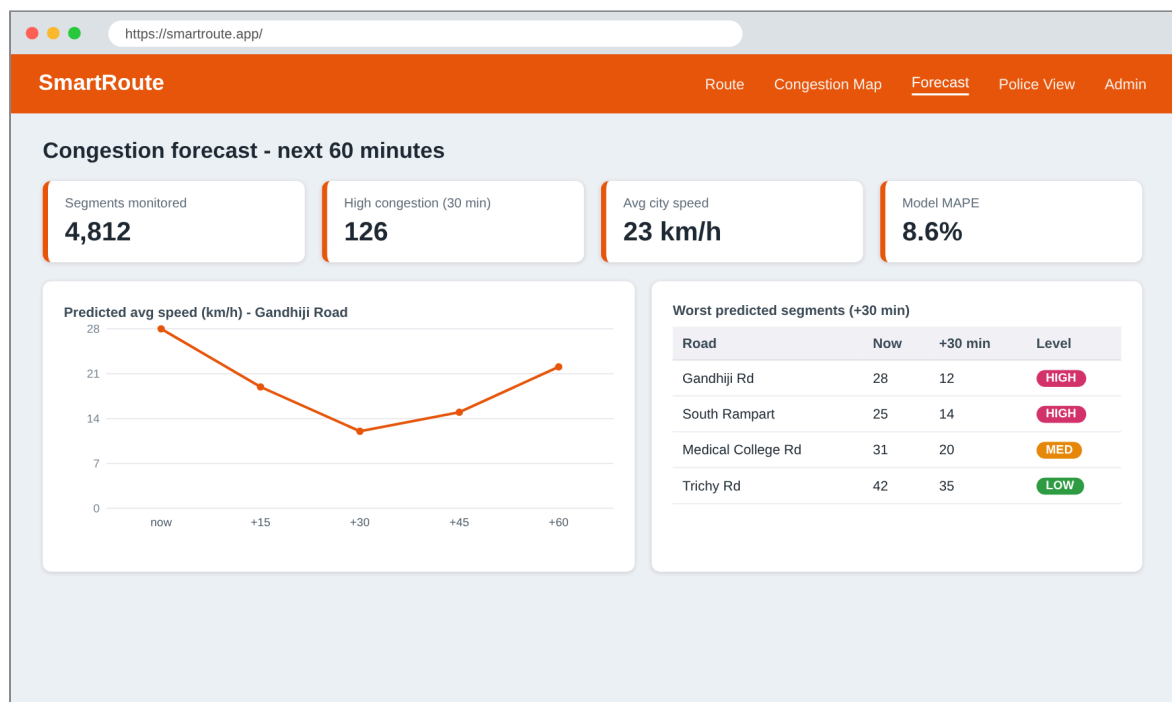
To the right of the form is a section titled "Today's alerts" which lists three alerts:

- School zone congestion expected 8:30 - 9:15 AM on Gandhiji Road.
- Rain expected after 4 PM - travel times +12%.
- Temple car festival on East Main Street - road closed.

RECOMMENDED ROUTE



CONGESTION FORECAST



TRAFFIC POLICE - HOTSPOTS

https://smartroute.app/

SmartRoute

[Route](#)[Congestion Map](#)[Forecast](#)[Police View](#)[Admin](#)

Predicted hotspots and deployment

Horizon: 30 min

Zone: Central

Junction	Peak time	Predicted delay	Suggestion
Old Bus Stand jn.	08:50	+14 min	Deploy 2
Rail station jn.	09:10	+9 min	Deploy 1
Gandhiji Rd / Hospital	08:40	+11 min	Deploy 2
Collector office jn.	10:00	+4 min	Monitor

CHAPTER VIII

CONCLUSION AND FUTURE ENHANCEMENT

8.1 Conclusion

The Real-Time Traffic Congestion Prediction and Route Optimizer predicts travel time on road segments up to 60 minutes ahead with a MAPE of 8.6% using an LSTM model, outperforming historical average and ARIMA baselines. Using these predictions as edge weights in a time-dependent A* search produces routes that avoid congestion before it forms, saving several minutes on typical peak-hour trips.

The police view converts the same predictions into actionable deployment suggestions. The system demonstrates how machine learning and graph algorithms together can make urban mobility smarter using only open data.

8.2 Future Enhancement

- Use graph neural networks (DCRNN / Graph WaveNet) to capture spatial dependencies.
- Adaptive traffic signal control using predicted flows.
- Integration with CCTV-based vehicle counting.
- Mobile app with voice navigation and push alerts.
- Carbon emission estimation for each route.

CHAPTER IX

BIBLIOGRAPHY

1. Williams, B. M., and Hoel, L. A., "Modeling and Forecasting Vehicular Traffic Flow as a Seasonal ARIMA Process", Journal of Transportation Engineering, Vol. 129, No. 6, 2003.
2. Ma, X., Tao, Z., Wang, Y., Yu, H., and Wang, Y., "Long short-term memory neural network for traffic speed prediction using remote microwave sensor data", Transportation Research Part C, Vol. 54, pp. 187-197, 2015.
3. Li, Y., Yu, R., Shahabi, C., and Liu, Y., "Diffusion Convolutional Recurrent Neural Network: Data-Driven Traffic Forecasting", Proc. ICLR, 2018.
4. Chen, T., and Guestrin, C., "XGBoost: A Scalable Tree Boosting System", Proc. ACM SIGKDD, pp. 785-794, 2016.
5. Hart, P. E., Nilsson, N. J., and Raphael, B., "A Formal Basis for the Heuristic Determination of Minimum Cost Paths", IEEE Transactions on Systems Science and Cybernetics, Vol. 4, No. 2, 1968.
6. Boeing, G., "OSMnx: New methods for acquiring, constructing, analyzing, and visualizing complex street networks", Computers, Environment and Urban Systems, Vol. 65, pp. 126-139, 2017.

WEBSITE REFERENCE

1. <https://osmnx.readthedocs.io>
2. <https://xgboost.readthedocs.io>
3. <https://leafletjs.com>
4. <https://www.openstreetmap.org>